ИС ЭСФ — Backend задачи

Предварительное условие (не код)

> ⚠️ **Без этого интеграция невозможна:**
>
> - Получить сертификат ЭЦП НУЦ РК (юридическое лицо COUBE)
> - Зарегистрироваться как интеграционный клиент КГД
> - Получить доступ к тестовому стенду
> - Контакт: esfsd@kgd.minfin.gov.kz | +7 (7172) 72-51-61

Текущее состояние

Что есть	Где
Kalkan (ЭЦП НУЦ РК)	`signature/` module — уже работает
Invoice entity + Service	`invoice/` module — полностью
Organization с BIN	`organization/model/Organization.java`
DocumentStatus enum	`invoice/entity/DocumentStatus.java`
ESF SDK v4.0.0	`coube-documentation/docs/ИС ЭСФ/esf-sdk-2025/`

Что нужно создать: новый модуль `esf/` в backend.

Flow работы (полная картина)

Coube — платформа. У каждого перевозчика своя ЭЦП. Приватный ключ никогда не покидает устройство перевозчика.

```
ПЕРЕВОЗЧИК                  COUBE BACKEND                    КГД (ИС ЭСФ)
(веб / мобилка)                   |                                |
       |                          |                                |
       |  1. Перевозка завершена,  |                                |
       |     АВР подписан обеими   |                                |
       |     сторонами             |                                |
       |                          |                                |
       |── GET /esf/invoices/{id}/auth-ticket ──────────────────►  |
       |                          |                                |
       |                  2. Формирует ESF XML                     |
       |                     (InvoiceToEsfMapper)                  |
       |                          |                                |
       |                  3. Запрашивает тикет авторизации         |
       |                     AuthService.createAuthTicket(ИИН) ──►|
       |                          |◄── authTicketXml ──────────────|
       |                          |                                |
       |◄── { authTicketXml, esfXml } ──────────────────────────── |
       |                          |                                |
       |  4. Подписывает authTicketXml своей ЭЦП                   |
       |     Подписывает esfXml своей ЭЦП                          |
       |     (NCALayer на вебе / NCAMobile на мобилке)             |
       |                          |                                |
       |── POST /esf/invoices/{id}/send ────────────────────────►  |
       |   { signedXml, signedAuthTicket }                         |
       |                          |                                |
       |                  5. Открывает сессию в КГД                |
       |                     createSessionSigned(BIN,              |
       |                       signedAuthTicket) ──────────────►|
       |                          |◄── sessionId ──────────────────|
       |                          |                                |
       |                  6. Отправляет ЭСФ                        |
       |                     sendInvoice(sessionId, signedXml) ──►|
       |                          |◄── registrationNumber ─────────|
       |                          |                                |
       |                  7. Закрывает сессию                      |
       |                     closeSession(sessionId) ────────────►|
       |                          |                                |
       |                  8. Сохраняет в esf_documents:            |
       |                     status = SENT                         |
       |                     registrationNumber = "..."            |
       |                          |                                |
       |◄── EsfStatusResponse ────|                                |
       |    (статус + рег. номер)  |                                |
```

```
      |                          |                              |
      |                  Scheduler (каждые 5 мин)               |
      |                  queryInvoiceStatus(regNumber) ──────────►|
      |                          |◄── DELIVERED ─────────────────|
      |                  status = DELIVERED                      |

── ── ── ── ── ── ── ОШИБКА ── ── ── ── ── ── ── ── ── ──

      |                          |◄── EsfApiException ───────────|
      |                  status = DECLINED                       |
      |                  errorCode + errorMessage сохранены      |
      |◄── ошибка с деталями ──────|                             |
```

Статусы EsfDocument

```
NOT_SENT → SENDING → SENT → DELIVERED
                   → DECLINED   (ошибка КГД)
         DELIVERED → CANCELLED  (отзыв)
```

Какой таск за что отвечает

Таск	Отвечает за
BE-1	ESF SDK подключён в проект
BE-2	Таблица `esf_documents` в БД
BE-3	Entity + Repository для EsfDocument
BE-4	`getAuthTicket` + `openSessionSigned` (шаги 3, 5, 7)
BE-5	Формирование ESF XML из Invoice (шаг 2)
BE-6	Основная логика отправки (шаги 5–8)
BE-7	API endpoints (`/auth-ticket` , `/send` , `/status` , `/cancel`)
BE-8	Scheduler синхронизации статусов

TASK-ESF-BE-1: Установить ESF SDK как локальные Maven зависимости

Приоритет: 🔴 Критический (блокирует весь ESF backend)

Что сделать

1. Установить jar-файлы в локальный Maven репозиторий:

```
# Из папки esf-sdk-2025/Документация BC SDK/sdk/maven/
chmod +x mvninstall.sh && ./mvninstall.sh
# Установит: esf-client:v4.0.0, vstore-client:v4.0.0, vstore-model:v4.0.0
```

2. Добавить в `coube-backend/build.gradle`:

```
dependencies {
    // ESF SDK
    implementation 'ru.uss.esf:esf-client:v4.0.0'
    implementation 'ru.uss.esf:vstore-client:v4.0.0'
    implementation 'ru.uss.esf:vstore-model:v4.0.0'
}
```

3. Или использовать локальный файловый репозиторий в gradle:

```
repositories {
    maven { url 'file://${System.getProperty("user.home")}/.m2/repository' }
}
```

Критерии готовности

- [ ] `./gradlew build` проходит без ошибок компиляции ESF классов
- [ ] Классы `ru.uss.vstore.*` доступны в проекте

TASK-ESF-BE-2: Миграция БД — таблица для ESF документов

Приоритет: 🔴 Критический
Зависит от: TASK-VAT-BE-1 (НДС должны быть готовы)

Что сделать

```sql
-- V{timestamp}__create_esf_documents.sql

CREATE TABLE applications.esf_documents (
    id                   BIGSERIAL PRIMARY KEY,
    invoice_id           BIGINT       NOT NULL REFERENCES applications.invoices(id),
    registration_number  VARCHAR(100) NULL,                -- рег. номер ЭСФ из КГД
    status               VARCHAR(50)  NOT NULL DEFAULT 'NOT_SENT',
    -- NOT_SENT | SENDING | SENT | DELIVERED | DECLINED | CANCELLED
    sent_at              TIMESTAMP    NULL,
    last_sync_at         TIMESTAMP    NULL,
    error_code           VARCHAR(100) NULL,
    error_message        TEXT         NULL,
    xml_body             TEXT         NULL,                -- сохранить для отладки
    created_at           TIMESTAMP    NOT NULL DEFAULT now(),
    updated_at           TIMESTAMP    NOT NULL DEFAULT now()
);

CREATE UNIQUE INDEX idx_esf_documents_invoice_id ON applications.esf_documents(invoice_

CREATE INDEX idx_esf_documents_status ON applications.esf_documents(status)
    WHERE status IN ('SENDING', 'SENT');

COMMENT ON TABLE applications.esf_documents IS 'Статусы и данные ЭСФ, отправленных в ИС
```

Критерии готовности

- ☐ Миграция применяется без ошибок
- ☐ Один invoice → один ESF документ (unique constraint)

TASK-ESF-BE-3: Entity и Repository для EsfDocument

Приоритет: 🔴 Критический
Зависит от: TASK-ESF-BE-2

Что сделать

1. Enum статусов:

```
// esf/entity/EsfStatus.java
public enum EsfStatus {
    NOT_SENT,   // Не отправлен
    SENDING,    // В процессе отправки
    SENT,       // Отправлен (ждём подтверждения)
    DELIVERED,  // Принят ИС ЭСФ
    DECLINED,   // Отклонён ИС ЭСФ (ошибка)
    CANCELLED   // Отозван
}
```

2. Entity:

```java
// esf/entity/EsfDocument.java
@Entity
@Table(name = "esf_documents", schema = "applications")
public class EsfDocument extends BaseIdEntity {

    @OneToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "invoice_id", nullable = false)
    private Invoice invoice;

    @Column(name = "registration_number")
    private String registrationNumber;

    @Enumerated(EnumType.STRING)
    @Column(name = "status", nullable = false)
    private EsfStatus status = EsfStatus.NOT_SENT;

    @Column(name = "sent_at")
    private LocalDateTime sentAt;

    @Column(name = "last_sync_at")
    private LocalDateTime lastSyncAt;

    @Column(name = "error_code")
    private String errorCode;

    @Column(name = "error_message", columnDefinition = "TEXT")
    private String errorMessage;

    @Column(name = "xml_body", columnDefinition = "TEXT")
    private String xmlBody;
}
```

3. Repository:

```java
// esf/repository/EsfDocumentRepository.java
public interface EsfDocumentRepository extends JpaRepository<EsfDocument, Long> {
    Optional<EsfDocument> findByInvoiceId(Long invoiceId);
    List<EsfDocument> findByStatus(EsfStatus status);
    List<EsfDocument> findByStatusIn(List<EsfStatus> statuses);
}
```

Критерии готовности

- [ ] Entity маппируется на таблицу
- [ ] Repository методы работают

TASK-ESF-BE-4: ESF Session Service (авторизация)

Приоритет: 🔴 Критический
Зависит от: TASK-ESF-BE-1

Контекст

Coube — платформа, у каждого перевозчика своя ЭЦП. Coube не хранит приватные ключи клиентов.

Используется `createSessionSigned` (КГД документация, раздел 2.1, метод 7):

- Бэкенд получает тикет от `AuthService.createAuthTicket`
- Перевозчик подписывает тикет своей ЭЦП на клиенте (NCALayer / NCAMobile)
- Бэкенд открывает сессию через подписанный тикет — приватный ключ на сервер не передаётся

Что сделать

1. Получить тикет для подписания (вызывается первым):

```java
// esf/client/EsfSessionService.java
// Используем @Flow — cross-module логика (esf + signature + invoice)
@Flow
@Slf4j
public class EsfSessionService {

    // Шаг 1: получить XML тикет для подписания на клиенте
    // Вызвать AuthService.createAuthTicket(iin) → вернуть authTicketXml
    public String getAuthTicket(String iin) {
        // AuthService.createAuthTicket(iin) → authTicketXml
    }

    // Шаг 2: открыть сессию через уже подписанный тикет
    // signedAuthTicket — XML Dsig, подписанный клиентом (NCALayer/NCAMobile)
    public String openSessionSigned(String tin, String signedAuthTicket) {
        // SessionService.createSessionSigned(tin, signedAuthTicket) → sessionId
    }

    // Закрыть сессию
    public void closeSession(String sessionId) {
        // SessionService.closeSession(sessionId)
    }
}
```

Конфиг (application.yml):

```yaml
esf:
  api:
    url: https://esf.gov.kz:8443/esf-web/ws/api1   # prod
    # url: https://esf.gov.kz:8080/esf-web/ws/api1  # test
```

> Сертификат Coube не нужен — сессия открывается от имени перевозчика через его подписанный тикет.

Критерии готовности

- [ ] `getAuthTicket(iin)` возвращает XML тикет от КГД
- [ ] `openSessionSigned(tin, signedAuthTicket)` успешно открывает сессию на тестовом стенде
- [ ] Закрывает сессию при любом исходе (finally)
- [ ] Логирует sessionId для отладки (без sensitive данных)

TASK-ESF-BE-5: Invoice → ESF XML маппер

Приоритет: 🔴 Критический
Зависит от: TASK-ESF-BE-4, TASK-VAT-BE-5

Что сделать

Маппинг из Invoice в XML формат ЭСФ (формат определён XSD из SDK).

```
// esf/mapper/InvoiceToEsfMapper.java
@Component
public class InvoiceToEsfMapper {

    public String toEsfXml(Invoice invoice) {
        // Маппинг полей:
        // invoice.executorOrganization.iinBin → seller.tin
        // invoice.customerOrganization.iinBin → customer.tin
        // invoice.invoiceNumber           → invoiceNum
        // invoice.documentDate            → invoiceDate
        // invoice.transportations[0].completedAt → turnoverDate
        // invoice.totalAmountWithoutVat   → product.priceWithoutTax (итого)
        // invoice.totalVatAmount          → product.ndsAmount
        // invoice.totalAmountWithVat      → product.priceWithTax
        // vatRate из transportation        → product.ndsRate (16%)

        // Услуга перевозки как одна строка (или по каждой перевозке):
        // product.name = "Услуги по перевозке грузов"
        // product.unit = "услуга" (код 896 - услуга)
        // product.qty = 1
    }
}
```

Важно: формат XML определён в XSD файлах SDK (`api-wsdl/xsd/`). Использовать классы из `vstore-model` для построения объектов, затем сериализовать в XML через JAXB.

Критерии готовности

- ☐ XML валидируется по XSD из SDK
- ☐ Поле `seller.tin` = BIN исполнителя (12 знаков)

- [ ] Поле `customer.tin` = BIN заказчика
- [ ] Суммы совпадают с Invoice (net + vat = gross)
- [ ] НДС 16% указан корректно (или "Без НДС" если vatAmount=0)

TASK-ESF-BE-6: ESF Sending Service

Приоритет: 🔴 Критический

Зависит от: TASK-ESF-BE-3, TASK-ESF-BE-4, TASK-ESF-BE-5

Что сделать

```java
// esf/service/EsfSendingService.java
// Используем @Flow — cross-module логика (esf + invoice + organization)
@Flow
@Transactional
@Slf4j
public class EsfSendingService {

    // signedXml         — ESF XML, подписанный перевозчиком на клиенте (NCALayer/NCAMo
    // signedAuthTicket  — тикет авторизации, подписанный перевозчиком на клиенте
    public EsfDocument sendInvoice(Long invoiceId, String signedXml, String signedAuthT
        Invoice invoice = invoiceRepository.findById(invoiceId)
            .orElseThrow(() -> new EntityNotFoundException("Invoice not found: " + invo

        // Проверки
        if (invoice.getStatus() != DocumentStatus.SIGNED) {
            throw new BusinessException("ЭСФ можно отправить только после подписания сч
        }

        EsfDocument esfDoc = esfDocumentRepository.findByInvoiceId(invoiceId)
            .orElse(EsfDocument.builder().invoice(invoice).build());

        if (esfDoc.getStatus() == EsfStatus.DELIVERED) {
            throw new BusinessException("ЭСФ уже был успешно отправлен");
        }

        String tin = invoice.getExecutorOrganization().getIinBin();
        String sessionId = null;
        try {
            esfDoc.setStatus(EsfStatus.SENDING);
            esfDoc.setXmlBody(signedXml);
            esfDocumentRepository.save(esfDoc);

            // Открываем сессию через подписанный тикет перевозчика
            sessionId = esfSessionService.openSessionSigned(tin, signedAuthTicket);

            String regNumber = esfApiClient.sendInvoice(sessionId, signedXml);

            esfDoc.setRegistrationNumber(regNumber);
            esfDoc.setStatus(EsfStatus.SENT);
            esfDoc.setSentAt(LocalDateTime.now());
```

```
        } catch (EsfApiException e) {
            esfDoc.setStatus(EsfStatus.DECLINED);
            esfDoc.setErrorCode(e.getErrorCode());
            esfDoc.setErrorMessage(e.getMessage());
            log.error("ESF sending failed for invoice {}: {}", invoiceId, e.getMessage(
        } finally {
            if (sessionId != null) esfSessionService.closeSession(sessionId);
        }

        esfDoc.setLastSyncAt(LocalDateTime.now());
        return esfDocumentRepository.save(esfDoc);
    }
}
```

Критерии готовности

- [ ] Нельзя отправить неподписанный счёт
- [ ] Нельзя отправить повторно DELIVERED
- [ ] При ошибке — статус DECLINED + сообщение об ошибке сохраняется
- [ ] Регистрационный номер сохраняется при успехе
- [ ] Логирование всех шагов

TASK-ESF-BE-7: API Endpoints для ESF операций

Приоритет: 🔴 Критический
Зависит от: TASK-ESF-BE-6

Что сделать

```
// esf/controller/EsfController.java
// Паттерн как в InvoiceController — @AuthorizationRequired + @RequireOrganizationHeade
@RestController
@RequestMapping("/api/v1/esf")
@RequiredArgsConstructor
@AuthorizationRequired(
    roles = { KeycloakRole.CEO, KeycloakRole.ACCOUNTANT, KeycloakRole.SIGNER },
    companyTypes = { CompanyType.EXECUTOR }  // только перевозчик выставляет ЭСФ
)
@RequireOrganizationHeader
public class EsfController {

    // Шаг 1: получить XML тикет для подписания на клиенте (NCALayer / NCAMobile)
    // Фронт/мобилка подписывают его ЭЦП перевозчика и передают в /send
    @GetMapping("/invoices/{invoiceId}/auth-ticket")
    public ResponseEntity<AuthTicketResponse> getAuthTicket(@PathVariable Long invoiceI

    // Шаг 2: отправить ЭСФ — принимает уже подписанные данные от клиента
    @PostMapping("/invoices/{invoiceId}/send")
    public ResponseEntity<EsfStatusResponse> sendInvoice(
        @PathVariable Long invoiceId,
        @RequestBody EsfSendRequest request  // signedXml + signedAuthTicket
    ) { ... }

    // Получить статус ЭСФ для счёта
    @GetMapping("/invoices/{invoiceId}/status")
    public ResponseEntity<EsfStatusResponse> getStatus(@PathVariable Long invoiceId) {

    // Отозвать ЭСФ
    @PostMapping("/invoices/{invoiceId}/cancel")
    public ResponseEntity<EsfStatusResponse> cancelInvoice(@PathVariable Long invoiceId
}
```

Request / Response DTO:

```java
// Запрос на отправку — клиент передаёт подписанные данные
public record EsfSendRequest(
    String signedXml,          // ESF XML подписанный ЭЦП перевозчика
    String signedAuthTicket    // Тикет авторизации подписанный ЭЦП перевозчика
) {}

// Тикет для подписания на клиенте
public record AuthTicketResponse(
    String authTicketXml,      // XML тикет от КГД — клиент подписывает и возвращает в
    String esfXml              // ESF XML — клиент подписывает и возвращает в /send
) {}

public record EsfStatusResponse(
    Long invoiceId,
    EsfStatus status,
    String registrationNumber,
    LocalDateTime sentAt,
    LocalDateTime lastSyncAt,
    String errorCode,
    String errorMessage
) {}
```

Также добавить ESF статус в InvoiceResponse:

```java
public record InvoiceResponse(
    // ...существующие поля...
    EsfStatusResponse esfStatus  // null если ЭСФ не отправлялся
) {}
```

Критерии готовности

- ☐ GET `/esf/invoices/{id}/auth-ticket` — возвращает XML тикет и ESF XML для подписания
- ☐ POST `/esf/invoices/{id}/send` — принимает signedXml + signedAuthTicket, отправляет в КГД
- ☐ GET `/esf/invoices/{id}/status` — возвращает текущий статус
- ☐ POST `/esf/invoices/{id}/cancel` — отзывает ЭСФ
- ☐ InvoiceResponse включает esfStatus

TASK-ESF-BE-8: Фоновая синхронизация статусов (Scheduler)

Приоритет: 🟡 Средний (Фаза 2)
Зависит от: TASK-ESF-BE-6

Что сделать

```
// esf/scheduler/EsfStatusSyncScheduler.java
@Component
@Slf4j
public class EsfStatusSyncScheduler {

    // Каждые 5 минут проверять статус SENT документов
    @Scheduled(fixedDelay = 5 * 60 * 1000)
    public void syncPendingStatuses() {
        List<EsfDocument> pendingDocs = esfDocumentRepository
            .findByStatusIn(List.of(EsfStatus.SENDING, EsfStatus.SENT));

        for (EsfDocument doc : pendingDocs) {
            try {
                String tin = doc.getInvoice().getExecutorOrganization().getBin();
                String currentStatus = esfSessionService.withSession(tin, sessionId ->
                    esfApiClient.queryInvoiceStatus(sessionId, doc.getRegistrationNumbe
                );
                // Обновить статус на основе ответа
            } catch (Exception e) {
                log.warn("Failed to sync ESF status for invoice {}", doc.getInvoice().g
            }
        }
    }
}
```

Ограничение ИС ЭСФ: не более 60 запросов/минуту. При большом количестве документов — добавить задержку между запросами.

Критерии готовности

- [ ] Планировщик запускается каждые 5 минут
- [ ] Не блокирует основной поток при ошибках

- [ ] Логирует изменения статусов
- [ ] Соблюдает rate limit ИС ЭСФ